

An explicit power saving for the exact discrete Fourier transform

OpenAI

September 25, 2026

Abstract

We give a deterministic algorithm that computes the discrete Fourier transform at every length n in $O(n(\log n)^{1-10^{-13}})$ operations. The model uses exact complex arithmetic, unrestricted coefficients, specified Fourier roots, and unit-cost logarithmic-size indexing; scalar preparation and array organization are included.

1 Introduction

For $d \geq 1$, put $\zeta_d = \exp(2\pi i/d)$ and let

$$F_d = (\zeta_d^{jk})_{0 \leq j, k < d}$$

be the unnormalized discrete Fourier matrix. Its fast computation is a basic instance of replacing a dense linear map by a short sequence of structured operations. The mixed-radix factorization of Cooley and Tukey [8] computes transforms at highly composite lengths in $O(n \log n)$ arithmetic operations. Good’s coprime-factor construction identifies certain one-dimensional transforms with tensor products, without intervening twiddle factors [9, Sections 11–12]; Cooley, Lewis, and Welch explain its relationship with Thomas’s construction and the mixed-radix FFT [7]. Bluestein’s chirp identity reduces an arbitrary Fourier length to convolution and, with an ordinary FFT, yields the $O(n \log n)$ arithmetic bound at every length [5]. Rabiner, Schafer, and Rader give the weighted-convolution derivation, padding, and operation count [14, Sections II–III].

Improvements within that scale and improvements of the scale itself are different questions. Alman and Rao use matrix non-rigidity to improve the Walsh–Hadamard transform and then the leading constant for power-of-two discrete Fourier transforms [3, Theorem 1.2 and Section 7]. Their Fourier bound has leading term $(15/4)n \log_2 n$ in a count of real arithmetic operations, with circuit structure and constants precomputed. Here we obtain a power saving in the logarithm at every length, in an exact complex-arithmetic model that also counts scalar preparation and array organization.

Lower bounds at the $n \log n$ scale depend on the permitted operations. Morgenstern’s determinant argument applies to the unnormalized Fourier transform with bounded coefficients [11]. Ailon’s entropy bound concerns the normalized transform on n coordinates with unitary two-coordinate gates [1, Theorem 2.1]; his later bounds also quantify restrictions on intermediate conditioning [2]. Our model permits unrestricted coefficients and exact intermediate values. The logarithmic word size below describes integer addresses; it does not limit the precision of complex registers.

The main obstacle is to turn a saving for one fixed finite matrix into a single algorithm for arbitrary Fourier lengths. A finite saving must survive three transitions: using all available coordinates during recursive batching, compiling small Fourier matrices without increasing their widths, and organizing a product of many coordinate sets with linear array work. The complexity of preparing the scalars

and constructing the schedules must also be counted. The theorem below handles these transitions explicitly.

Two companion results clarify the starting point. The finite tensor saving and the nonuniform subsequence theorem in *Finite tensor savings and exact Fourier circuits* [12] have an independent proof through matrix prices and exact finite identities. The complex phase network in *Integer multiplication below $n \log n$* [13, Proposition 8 and Section 3.5] provides a particular two-coordinate kernel with a strict saving. We reproduce the network needed here and supply its uniform array implementation. The integer-multiplication article's arithmetic model and its later tape and precision arguments are separate from the model used in this paper.

1.1 Arithmetic model and main result

Our model has complex registers and integer address registers. A complex field operation has unit cost. In the computation on the input vector, only addition, subtraction, and multiplication by prepared, input-independent scalars are used. Every such operation is charged. Division is used in scalar preparation only, with a proof that its denominator is nonzero. Integer operations and random access on $O(\log(n+2))$ -bit words also have unit cost; a fixed number of such words may represent an address or integer. All constants implicit in these conventions are independent of n .

We supply an explicitly specified Fourier root depending on n . Equivalently, a root-of-unity operation returns the specified ζ_d , at unit cost, for a stated order d . The theorem below needs only one such root, of polynomial order. Preparing other scalars from it, constructing the schedule, and organizing the arrays are included in the running time. We do not assume that field operations on rational constants can generate roots of arbitrarily large order. There is no bound on complex coefficients or intermediate magnitudes, and no finite-precision or numerical-stability assertion.

Define the absolute constants

$$m = 10^6, \quad W_* = 2^{71}, \quad \Delta = 6871402692000000, \quad \lambda = m - \frac{\Delta}{W_*}, \quad \theta = \log_m \lambda. \quad (1.1)$$

The construction in Section 2 uses blocks of m binary coordinates and W_* arrays. The saving Δ gives the recurrence multiplier λ and hence the exponent θ . These constants satisfy $1 < \lambda < m$ and

$$\theta = 0.9999999999978935615699598 \dots < 1.$$

The decimal is included only to indicate the size of the saving; the definition in (1.1) is exact.

Theorem 1.1. *There is a single deterministic algorithm which, for every positive integer n and every $x \in \mathbb{C}^n$, computes $F_n x$ exactly. In the model above, including scalar preparation, schedule construction, and index work, its running time is*

$$O\left(n(\log n)^\theta (\log \log n)^{4-\theta}\right) \quad (1.2)$$

as $n \rightarrow \infty$. It suffices to supply the specified root ζ_{D_*} for an explicitly computable integer $D_* < 1024n^3$.

Corollary 1.2. *In the same model, the transform is computable at every length in*

$$O\left(n(\log n)^{1-10^{-13}}\right) = o(n \log n)$$

time.

A finite initial range is handled by the direct formula for F_n . The constants in the algorithm are enormous, so these are asymptotic existence and explicitness results, with no useful crossover estimate. The hypotheses on coefficients and exact roots are part of the result. In particular, the conclusion does not assert a corresponding improvement for stable floating-point FFTs or for bit complexity.

Corollary 1.3 (Exact convolution and polynomial multiplication). *There is a single deterministic algorithm which, for every positive integer n and every $a, b \in \mathbb{C}^n$, computes the linear convolution*

$$c_k = \sum_{\substack{0 \leq r, s < n \\ r+s=k}} a_r b_s, \quad 0 \leq k \leq 2n-2.$$

Equivalently, it multiplies two complex polynomials of degree less than n . In the exact complex field-operation, specified-root, and address cost model above, with general field multiplication permitted for the pointwise products, the total running time, including scalar preparation, schedule construction, padding, and index work, is

$$O\left(n(\log n)^\theta (\log \log n)^{4-\theta}\right) \quad (n \rightarrow \infty),$$

and hence is eventually $O(n(\log n)^{1-10^{-13}})$. This is a bilinear arithmetic algorithm, not an input-linear circuit: its pointwise products multiply a value linear in a by a value linear in b .

The proof in Section 5.5 uses three transforms at length $2n-1$, with one specified root reused throughout.

The corollary retains the exact-root and unrestricted-coefficient assumptions. It is not a result about integer bit complexity, bounded-coefficient circuits, numerical stability, finite fields, or formal power-series algorithms.

1.2 The construction and its reusable parts

Two constructions feed a common tensor schedule, which then supplies the transform at every length.

1. A fixed network gives a fast algorithm for tensor powers of

$$C = \frac{1}{2} \begin{pmatrix} 1+i & 1-i \\ 1-i & 1+i \end{pmatrix}.$$

Its binary labels prescribe changes of frame between scalar gates. The completed network transforms every physical role, including roles used as temporary storage. Padding the number of roles to a power of two makes recursive batching exact. The resulting recurrence gives $C^{\otimes k}$ in $O(2^k(k+1)^\theta)$ time.

2. Independently, Newton interpolation factors every Fourier matrix F_r into diagonal factors and a triangular Toeplitz matrix and its transpose. We compile these into a word of $O(1 + \log^4 r)$ layers on *exactly* r coordinates. Borrowed coordinates may initially contain arbitrary data and are restored. The compiler chooses chunk sizes by counting the registers required by its own convolution programs; all local compilation and scalar preparation take polynomial time in r .

3. Synchronize these words on many coprime axes. Each simultaneous pair layer decomposes into sectors carrying tensor powers of C . A concrete sector-address formula packs all sectors in linear array work. Products of small odd primes, supplemented by one power of two, give suitable working lengths within a factor of two of $2n$. A chirp convolution then gives every requested length.

The role-preserving tensor recurrence, the measured-fit local compiler, and the linear-time sector traversal are useful independently of the final choice of Fourier lengths. Together they prevent three possible losses: allocating fresh zero workspace at every recursive use, silently treating scalar preparation as free, and spending one operation per tensor axis at every array entry. The sharper logarithmic factor in (1.2) comes from the elementary estimate that the number of odd-prime factors is of order $\log n / \log \log n$. The appendix supplies two further general constructions: finite rational-algebra extraction from polynomial certificates, and bounded-printer uniformization of effectively generated transform circuits.

Figure 1 records the interfaces between these steps.

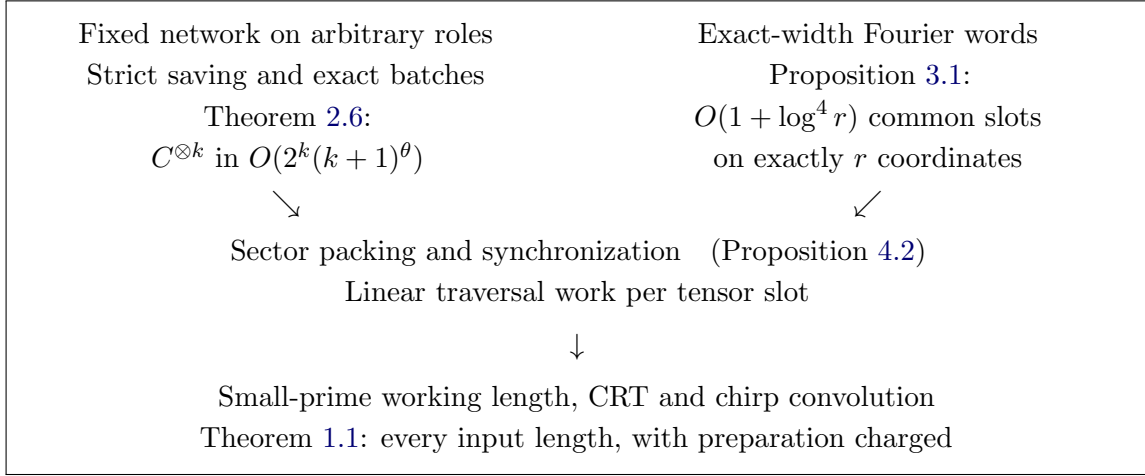


Figure 1: The quantitative proof. The tensor-power algorithm and the exact-width Fourier words are separate inputs to synchronization. Both constructions account for arbitrary data in their auxiliary roles. Appendix A gives the separate certificate-and-printer route.

Section 2 develops the fixed network. Section 3 gives the local Fourier compiler, Section 4 handles tensor scheduling and addresses, and Section 5 proves Theorem 1.1 and its convolution consequence. Appendix A gives a complementary effective route from finite algebraic certificates: it makes finite-win certificates uniform by exact rational algebra and supplies a local certificate that proves the search terminates. The general finite-win and generation results of [12] give a second source of the same certificate interface. That route is not needed for the exponent in (1.2).

The finite network is credited at its use to [13], and the exact-width Fourier mechanism is shared with [12, Section 3]. We prove the interfaces used here, including their preparation and indexing bounds. In particular, we do not import the later tape or precision bounds of [13], nor infer uniformity from the circuit-existence conclusion of [12].

2 An explicit saving for a fixed two-coordinate transform

Put

$$C = \begin{pmatrix} a & b \\ b & a \end{pmatrix}, \quad a = \frac{1+i}{2}, \quad b = \frac{1-i}{2}. \quad (2.1)$$

The ordinary algorithm applies the k factors of $C^{\otimes k}$ separately, using $O(k2^k)$ operations. We obtain a smaller exponent of k by a fixed network from the complex construction of [13, Proposition 8 and Section 3.5]. The network exchanges two banks and restores arbitrary auxiliary values. Changing the coordinate frame along its wires then transforms *every* bank and auxiliary array. The total dimension of the frame changes is smaller than the cost of transforming the arrays separately. We give the complete finite construction because this strict saving and the treatment of arbitrary auxiliaries are the main inputs to the recursion.

2.1 A scalar network with arbitrary auxiliary values

Fix

$$h = 100, \quad \mathcal{T} = \{T \subseteq \{1, \dots, h\} : |T| = 3\}, \quad v = \binom{h}{3} = 161700, \quad N_0 = v^3, \quad m = h^3 = 10^6. \quad (2.2)$$

There are two banks of scalar wires X_d, Y_d , indexed by $d = (d_1, d_2, d_3) \in \mathcal{T}^3$. Two triples are *neighbors* when they have even intersection; neighboring triples are distinct. The network has three stages. At stage j , fix the two indices other than d_j and operate on the two lists obtained by varying d_j . There are $I_0 = 3v^2$ such invocations. Each has its own auxiliary wires, distinct from all other invocations: one side wire A_{ST} for each *ordered* neighboring pair (S, T) , and $h + 1$ central wires $c_1, \dots, c_h, c_*$.

For one invocation, call the logical source and target banks $x = (x_T)$ and $y = (y_S)$. Define the fixed linear maps

$$\begin{aligned} (Vx)_{ST} &= x_T, & (Gx)_j &= \sum_{T \ni j} x_T, & (Gx)_* &= \sum_T x_T, \\ (JA)_S &= - \sum_{T: T \text{ neighbors } S} \frac{|S \cap T| - 1}{2} A_{ST}, & (Rc)_S &= \frac{\sum_{j \in S} c_j - c_*}{2}. \end{aligned}$$

The forward invocation consists of the following eight rows, in order:

row	update	gate grouping
0	$y \leftarrow y - JA$	one gate per target and its side wires
1	$y \leftarrow y - Rc$	one gate on the targets and center
2	$A \leftarrow A + Vx$	one gate per source and its side wires
3	$c \leftarrow c + Gx$	one gate on the sources and center
4	$y \leftarrow y + Rc$	one gate on the targets and center
5	$y \leftarrow y + JA$	one gate per target and its side wires
6	$c \leftarrow c - Gx$	one gate on the sources and center
7	$A \leftarrow A - Vx$	one gate per source and its side wires.

(2.3)

A gate is simply the indicated finite linear map on its wires. Its scalar implementation, including all additions and multiplications, has a fixed finite cost.

Lemma 2.1. *For arbitrary initial auxiliary values, the forward invocation sends (x, y, A, c) to $(x, y + x, A, c)$. Forward invocations from X to Y at stages 1 and 3, and the inverse forward invocation from Y to X at stage 2, give*

$$(X_d, Y_d) \mapsto (-Y_d, X_d) \quad (d \in \mathcal{T}^3)$$

and restore every auxiliary value.

Proof. The first two rows subtract $JA + Rc$ from y . The middle rows add $R(c + Gx) + J(A + Vx)$, and the last two restore c and A . The net addition is $(RG + JV)x$. The coefficient from x_T to y_S in RG is $(|S \cap T| - 1)/2$. It is 1 for $S = T$, 0 when the intersection has size 1, and is canceled by JV when the intersection has size 0 or 2. Hence $RG + JV = I$. The three stages therefore send (x, y) to $(x, y + x)$, then $(-y, y + x)$, then $(-y, x)$. The auxiliary values cancel in each invocation separately. $\square$

A triple has

$$d_{\text{nbr}} = \binom{97}{3} + 3 \cdot 97 = 147731$$

neighbors: the two terms count intersections of size 0 and 2. Consequently the number of physical wires is

$$W = 2N_0 + I_0(vd_{\text{nbr}} + 101) = 1873807244643542670000. \quad (2.4)$$

The factor vd_{nbr} counts ordered pairs, as required by the schedule.

2.2 Frames turn binary dimensions into directional operations

We now explain how to turn the scalar exchange into a tensor transform. An edge of the scalar network connects consecutive touches of a physical wire, including its source and sink. We will assign a binary subspace to each gate and terminal, with one common subspace for all wires touched by a gate. This subspace specifies a change of frame on the complex array replacing each wire. The construction below has two useful properties: common frames commute with scalar gates, and moving between suitable nested subspaces costs one directional operation per added or removed binary dimension.

For this purpose use the binary space $\mathcal{D} = \mathbb{F}_2^m$ with its ordinary dot product. A subspace is *nondegenerate* if its restricted bilinear form has zero radical. Its orthogonal complement and all projections below are taken over $\mathbb{F}_2$; these are separate from the complex spaces carrying the data.

For $z \in \mathbb{F}_2^m$, write R_z for translation of array addresses: $(R_z f)(x) = f(x + z)$. The normalized Walsh matrix

$$J_m = 2^{-m/2} ((-1)^{x \cdot y})_{x, y \in \mathbb{F}_2^m}$$

satisfies $J_m^2 = I$. Conjugating the diagonal character $(-1)^{z \cdot x}$ by J_m gives R_z , by cancellation of nontrivial binary characters. For a nondegenerate label U , let P_U be projection onto U along $U^\perp$, and set

$$q_U(x) = \text{wt}(P_U x) \pmod{4}, \quad \Phi_U = J_m \text{diag}_x(i^{q_U(x)}) J_m. \quad (2.5)$$

Here wt is integer Hamming weight. These matrices prove identities; the algorithm does not execute Walsh transforms. In particular $\Phi_0 = I$ and $\Phi_{\mathcal{D}} = C^{\otimes m}$.

Lemma 2.2. *Let $U, V \subseteq \mathcal{D}$ be nested nondegenerate subspaces. Call the orthogonal complement of the smaller subspace inside the larger their residual. Suppose it has an orthonormal binary basis of size ρ . Then $\Phi_V \Phi_U^{-1}$ is a product of exactly ρ directional kernels of the form $C_z = aI + bR_z$ or C_z^{-1} , where $z \neq 0$. Each inverse requires one forward kernel and a translation.*

Proof. For binary vectors x, y ,

$$\text{wt}(x + y) = \text{wt}(x) + \text{wt}(y) - 2|\text{supp}(x) \cap \text{supp}(y)|.$$

Thus weight is additive modulo four for orthogonal vectors. If $V = U \perp E$ and $z_1, \dots, z_\rho$ is an orthonormal basis of E , then

$$q_V(x) - q_U(x) = \sum_{j=1}^{\rho} \text{wt}(z_j)[z_j \cdot x] \pmod{4},$$

where brackets mean the integer representative 0 or 1. Every $\text{wt}(z_j)$ is odd, so its value modulo four is 1 or -1 . For a decreasing edge the entire expression is negated. The identity $i^{[z \cdot x]} = a + b(-1)^{z \cdot x}$ and Walsh conjugation give the stated kernels. Finally $R_z^2 = I$ and $C_z^2 = R_z$, whence $C_z^{-1} = R_z C_z$. $\square$

To apply this calculation to the network, replace each scalar wire by an array indexed by $\mathcal{D}$. Give every gate a nondegenerate label U , common to all wires it touches, and give each source and sink its own label. Insert $\Phi_V \Phi_U^{-1}$ along an edge from label U to label V , and apply the scalar gates pointwise. A scalar gate mixes its touched arrays with scalar coefficients, whereas their common frame acts on the address coordinate. These two operations commute. The intermediate frames therefore cancel, leaving the scalar network between its source and sink frames.

More generally, for an integer $f \geq 1$, index every array by f columns of m bits and use $\Phi_U^{\otimes f}$ as its frame. The same cancellation gives the complete linear map

$$\text{diag}_{\text{sinks}}(\Phi_U^{\otimes f}) \mathcal{S} \text{diag}_{\text{sources}}((\Phi_U^{-1})^{\otimes f}), \quad (2.6)$$

where $\mathcal{S}$ is the pointwise signed bank exchange of Lemma 2.1, acting as the identity on auxiliary roles. This is an identity on arbitrary values in every physical role. The remaining task is to choose labels whose terminal frame ratios give the desired transform and whose total residual dimension is less than Wm . Lemma 2.2 will then convert that dimension saving into fewer directional operations.

2.3 Binary labels and the dimension budget

Identify $\mathcal{D}$ with $D^{\otimes 3}$, where $D = \mathbb{F}_2^h$; the tensor dot product is the ordinary dot product on $\mathbb{F}_2^m$. For $T \in \mathcal{T}$, let t_T be its indicator. Then $t_T \cdot t_T = 1$, and neighboring indicators are orthogonal. Put

$$u_d = t_{d_1} \otimes t_{d_2} \otimes t_{d_3}, \quad U_d = \langle u_d \rangle.$$

Each u_d has norm one, so U_d is nondegenerate.

Start with the terminal labels. Sources have label 0, except that X_d has label U_d ; sinks have label $\mathcal{D}$, except that Y_d has label $U_d^\perp$. A route with terminal labels 0 and $\mathcal{D}$ has frame ratio $C^{\otimes m}$. The exceptional route $X_d \rightarrow Y_d$ will have the same ratio up to a translation, as we verify below. The total increase of terminal dimensions is $Wm - 2N_0$. Thus the margin below Wm is $2N_0$ before we charge twice the dimensions lost at any internal decreases.

At an invocation in stage j , write

$$E = D^{\otimes(j-1)}, \quad P = \left\langle \bigotimes_{l < j} t_{d_l} \right\rangle, \quad B = P^\perp \subset E, \quad Q = \left\langle \bigotimes_{l > j} t_{d_l} \right\rangle.$$

An empty tensor product is the full one-dimensional binary space. Thus $E = B \perp P$, with $B = 0$ in stage 1, and Q is the full future space in stage 3. For a physical bank entry d , write $t = t_{d_j}$.

The stage expands its current tensor factor from a triple line to all of D , while retaining the future line Q . More precisely, its incoming labels are $(E \otimes \langle t \rangle) \otimes Q$ on X and $(B \otimes \langle t \rangle) \otimes Q$ on Y ; its outgoing labels are $(E \otimes D) \otimes Q$ on X and

$$((B \otimes D) \perp (P \otimes t^\perp)) \otimes Q = (P \otimes \langle t \rangle)_{E \otimes D}^\perp \otimes Q$$

on Y . Separating the next triple from Q makes these exactly the next stage's incoming labels. At the first and last stages they agree with the specified terminal labels. We may insert these stage boundaries as extra vertices without changing any edge operation.

The following gate labels realize those boundary conditions. In the table each label is further tensored with Q , and t is the indicator at the physical bank entry of an entry-specific gate. The X labels increase to $E \otimes D$, while the Y labels increase through $B \otimes D$. A central wire must visit both levels: its return from $E \otimes D$ in row 3 to $B \otimes D$ in row 4 is the only decrease that will consume part of the margin.

row	physical wires touched	label before tensoring with Q
0	Y , side	$B \otimes \langle t \rangle$
1	Y , center	$B \otimes D$
2	X , side	$(B \otimes D) \perp (P \otimes \langle t \rangle)$
3	X , center	$E \otimes D$
4	Y , center	$B \otimes D$
5	Y , side	$(B \otimes D) \perp (P \otimes t^\perp)$
6	X , center	$E \otimes D$
7	X , side	$E \otimes D$.

(2.7)

The row numbers here are chronological, including in stage 2. At that stage, reverse the eight scalar updates and their signs, with logical source Y and target X . The physical touch sequence is Y -side, Y -center, X -side, X -center, Y -center, Y -side, X -center, X -side, exactly as in the table. Relabeling the ordered side pairs accordingly leaves their two physical triples neighboring. Every displayed label is nondegenerate.

Lemma 2.3. *Every edge has nested nondegenerate endpoint labels. Each nonzero orthogonal residual, meaning the complement of the smaller label inside the larger, has an orthonormal binary basis. If s is the sum of the residual dimensions, then*

$$s = Wm - 2N_0 + 2I_0(101)100 = 1873807244636671267308000000, \quad (2.8)$$

$$\Delta := Wm - s = 6871402692000000 > 0. \quad (2.9)$$

Proof. Here is the full list of nonzero or potentially nonzero residuals. All rows except the last are tensored with Q . The vectors t_X, t_Y on a side wire are its two physical triple indicators. The

omitted edges have zero residual.

wire and edge	orthogonal residual
$X : \text{in} \rightarrow 2$	$B \otimes t_X^\perp$
$X : 2 \rightarrow 3$	$P \otimes t_X^\perp$
$Y : 0 \rightarrow 1$	$B \otimes t_Y^\perp$
$Y : 4 \rightarrow 5$	$P \otimes t_Y^\perp$
center : source $\rightarrow 1$	$B \otimes D$
center : $1 \rightarrow 3, 3 \rightarrow 4, 4 \rightarrow 6$	$P \otimes D$ (each)
side : source $\rightarrow 0$	$B \otimes \langle t_Y \rangle$
side : $0 \rightarrow 2$	$(B \otimes t_Y^\perp) \perp (P \otimes \langle t_X \rangle)$
side : $2 \rightarrow 5$	$P \otimes \langle t_X, t_Y \rangle^\perp$
side : $5 \rightarrow 7$	$P \otimes \langle t_Y \rangle$
auxiliary : last $\rightarrow$ sink	$E \otimes D \otimes Q^\perp$.

In the last row $Q^\perp$ is taken in the full future tensor space. The side edge $2 \rightarrow 5$ explains the use of neighboring triples. Because $t_X \cdot t_Y = 0$ and both vectors have norm one,

$$t_Y^\perp = \langle t_X \rangle \perp \langle t_X, t_Y \rangle^\perp.$$

Its row-2 label $(B \otimes D) \perp (P \otimes \langle t_X \rangle)$ therefore lies in its row-5 label $(B \otimes D) \perp (P \otimes t_Y^\perp)$, with precisely the displayed residual. The other rows follow by $E = B \perp P$ and $D = \langle t \rangle \perp t^\perp$. These decompositions verify all inclusions and show that the only decreases are the central edges $3 \rightarrow 4$, each of dimension h .

The signed changes telescope along physical wires. Source dimensions sum to N_0 and sink dimensions to $Wm - N_0$, so their total is $Wm - 2N_0$. Each of the $I_0(101)$ central wires loses $h = 100$ dimensions once. Replacing signed changes by absolute changes adds twice these losses, proving the formula for s . In particular,

$$\Delta = 2N_0 - 2I_0(h+1)h = 2v^2(v - 3h(h+1)) > 0,$$

since $v > 3h(h+1)$. Substituting $h = 100$ and $v = 161700$ gives the displayed integers.

Nondegeneracy alone would not guarantee an orthonormal basis in characteristic two: a nondegenerate form may be alternating. Here every nonzero residual contains a vector of norm one. Indeed, $t^\perp$ and $\langle t_X, t_Y \rangle^\perp$ contain coordinate units outside supports of size at most 3 and 6 respectively. If nonzero, B contains a coordinate unit outside a support of size $3^{j-1} < h^{j-1}$, and the same argument applies to nonzero $Q^\perp$. Full spaces and the lines $P, Q, \langle t \rangle$ also contain norm-one vectors. Taking tensor products and choosing a nonzero summand in an orthogonal sum proves the claim for every residual in the table.

For completeness, a nondegenerate binary symmetric space with a norm-one vector has an orthonormal basis. Split off unit lines until the remaining space is alternating. A nonzero nondegenerate alternating space splits into orthogonal planes with bases p, q satisfying $p \cdot p = q \cdot q = 0$ and $p \cdot q = 1$: choose $p \neq 0$, choose q by nondegeneracy, and split off their plane. Retain a unit vector w from a line already removed. A plane perpendicular to w is absorbed by replacing (w, p, q) with

$$w + p, \quad w + q, \quad w + p + q.$$

These are pairwise orthogonal unit vectors and span the same three-dimensional space. Repetition absorbs all alternating planes. All choices can be made by finite binary linear algebra with a fixed ordering of coordinates and pivots. $\square$

2.4 Transforming every physical role

Proposition 2.4 (Finite network interface). *For every integer $f \geq 1$, the explicit network applies $C^{\otimes mf}$ to each of W arbitrary arrays indexed by $\mathbb{F}_2^{mf}$. It uses s directional steps, each on one physical array and conjugate by address permutations to copies of $C^{\otimes f}$ on fibers. All other work consists of a fixed number of pointwise linear maps and address permutations. The same assertion holds with any spectator address bits.*

Proof. Use the gate and terminal labels just constructed in (2.6), with each array address viewed as f columns of m bits. It remains to calculate the terminal frame ratios and implement the edge changes.

Except for the route $X_d \rightarrow Y_d$, a source and its routed sink have labels 0 and $\mathcal{D}$, so their frame ratio is $C^{\otimes mf}$. For the exceptional route put $u = u_d$. Since $u \cdot u = 1$, $P_{U_d}x = u[u \cdot x]$, and orthogonal weight additivity gives

$$q_{U_d^\perp}(x) - q_{U_d}(x) = \text{wt}(x) - 2 \text{wt}(u)[u \cdot x] = \text{wt}(x) + 2[u \cdot x] \pmod{4}.$$

Here $\text{wt}(u) = 27$ is odd. Hence the exceptional frame ratio is $R_u C^{\otimes m}$ in each column. Correct these translations at the sinks and undo the signed bank exchange. The result is $C^{\otimes mf}$ on each physical role, including every auxiliary array.

By Lemma 2.2, an edge with residual dimension ρ uses ρ products $C_z^{\otimes f}$ or $(C_z^{-1})^{\otimes f}$. Extend z to a binary basis and apply the corresponding invertible address change in each column. Translation by z becomes a flip of one selected bit, so the forward product is $C^{\otimes f}$ on those f selected bits, for each setting of all other bits. Inverse products add only translations. Summing the residual dimensions gives precisely s directional steps. The network, all its bases, and the number of remaining pointwise operations are fixed independently of f . Spectator bits merely index additional copies of these identities. $\square$

2.5 Linear-time address changes

The address permutations in Proposition 2.4 must not introduce a factor f . The following enumeration also explains the uniform construction of the recursive batches.

Lemma 2.5. *Let $k = mf + r$ with $0 \leq r < m$, and fix an invertible binary map on an m -bit column. Applying it in all f columns, translating columns by prescribed fixed vectors, and collecting one chosen output bit from every column into a contiguous f -bit address field can be implemented in $O(2^k)$ time. This includes all index preparation and data movement; the constant may depend on m and the fixed maps, but not on f or k .*

Proof. An m -bit column is a digit of radix 2^m . Since m and the maps are fixed, a finite table gives, for each possible column digit, its old bits, transformed bits, selected output bit, and remaining output bits. This table can be constructed once by the specified binary arithmetic; its cost is an absolute constant. The r spectator bits form one additional digit of radix 2^r , omitted when $r = 0$.

For a given k , compute the input and output bit-position strides and the displaced table contributions for each column. This costs $O(f + 1)$ operations with a constant depending on m . An input address and the desired output address are each sums of the contributions of their column digits and the spectator digit; the output contributions put the selected bits in their common field and all other bits outside it. Traverse the digit prefix tree, retaining these two partial sums. Extending a prefix by a digit adds its two tabulated contributions in constant time. At a leaf, copy the value between the two resulting addresses, using a separate output array to avoid overwriting

unread values. Every internal node has at least two children. There are 2^k leaves, so fewer than 2^{k+1} nodes, including the root. The preparation cost is also $O(2^k)$. Reversing the source and destination addresses implements the inverse permutation with the same bound.

Take the selected f bits as the least significant output field. Each fiber is then contiguous, and consecutive groups of W_* fibers, for any fixed W_* , are consecutive blocks of W_*2^f entries. No per-entry scan of the columns is needed to form or undo these groups. Addresses have $k + O(1)$ bits, with the fixed network sizes included in the constant. $\square$

2.6 Exact batching and the critical exponent

The least power of two at least W is

$$W_* = 2^{71}, \quad S = s + (W_* - W)m = W_*m - \Delta, \quad \lambda = \frac{S}{W_*} = m - \frac{\Delta}{2^{71}}, \quad \theta = \log_m \lambda. \quad (2.10)$$

In particular $1 < \lambda < m$, so $0 < \theta < 1$. The $W_* - W$ extra roles each use the m ordinary coordinate directions, preserving the absolute saving Δ .

Theorem 2.6. *There is a deterministic exact algorithm applying $C^{\otimes k}$ to an arbitrary complex array of length 2^k in*

$$O(2^k(k+1)^\theta)$$

time, for every integer $k \geq 0$. The bound includes fixed-table construction, all scalar arithmetic, array initialization, index preparation, and data movement. Only the rational constants and i are required.

Proof. We first transform W_* arbitrary arrays simultaneously. Set $K = m(71 + 1)$. For $k < K$, apply the k factors of C one at a time along their fibers. Because K is fixed, the cost divided by W_*2^k is bounded by an absolute constant throughout this base range, including $k = 0$.

For $k \geq K$, write $k = mf + r$, where $f = \lfloor k/m \rfloor$ and $0 \leq r < m$. Apply the r remaining bit factors directly. On W arrays use Proposition 2.4 with f columns and r spectator bits. On each of the $W_* - W$ remaining arrays use one directional step for each of the m coordinate positions, again simultaneously over all f columns. There are S directional steps in total.

Each such step requires 2^{k-f} independent transforms $C^{\otimes f}$. Since $f \geq 72$ and $k - f \geq (m - 1)f \geq 71$, the number 2^{k-f} is divisible by $W_* = 2^{71}$. After the permutation of Lemma 2.5, partition its fibers into exactly $2^{k-f}/W_*$ batches and use the present W_* -array algorithm recursively on each batch. No partially filled recursive batch occurs. Its correctness follows by induction on k , since $f < k$, and because the network interface applies to arbitrary values on *all* its roles.

Let $T(k)$ be the cost for W_* arrays and $t(k) = T(k)/(W_*2^k)$. The fixed number of pointwise gates, permutations, terminal corrections, and at most $m - 1$ leftover bit layers cost at most AW_*2^k outside recursive calls, for an absolute constant A . Batch setup and returning results are included in this bound: the batches partition a packed array, and their number is at most its length. Thus

$$\begin{aligned} T(k) &\leq AW_*2^k + S \frac{2^{k-f}}{W_*} T(f), \\ t(k) &\leq A + \lambda t(\lfloor k/m \rfloor) \quad (k \geq K). \end{aligned}$$

This exact recurrence permits the critical exponent $\theta = \log_m \lambda$. To see this directly, unroll it for d steps until the argument falls below K . Successive arguments are $\lfloor k/m^j \rfloor$, and $d \leq \lceil \log_m(k + 1) \rceil$. If B bounds the normalized base costs, then

$$t(k) \leq A \sum_{j=0}^{d-1} \lambda^j + B\lambda^d \leq \left(\frac{A}{\lambda - 1} + B \right) \lambda^d = O((k + 1)^\theta).$$

Here $\lambda^d \leq \lambda(k+1)^\theta$. The same estimate holds in the base range by increasing the constant.

Fixed tables may be generated once and reused; even generating them on the first invocation contributes an absolute constant. The recursive calls can run serially and reuse their working arrays. Their address fields, counters, and strides use $O(k+1)$ bits: at each active level only a fixed number of arrays of the current length is needed, and the lengths shrink from 2^k to $2^{\lfloor k/m \rfloor}$ and so on. The recursion stack has $O(\log(k+2))$ levels. Thus the claimed workspace is bounded by a constant times $\sum_{j=0}^d W_* 2^{\lfloor k/m^j \rfloor} + O(k+1) = O(W_* 2^k)$; consecutive nonterminal array lengths decrease by at least a factor of two. The claimed index costs therefore use exactly the logarithmic-word model of the paper.

Finally, initialize $W_* - 1$ unused arrays to zero, put the requested array in the remaining role, run the simultaneous procedure, and retain that role's output. Initialization costs $O(W_* 2^k) = O(2^k)$, and W_* is an absolute constant. This proves the stated single-array bound. $\square$

Remark 2.7. Restoring arbitrary auxiliary values in Lemma 2.1 is essential. After the frame changes, those physical roles are required to undergo the same tensor transform as the data banks. A network valid only with zero auxiliary values would not justify using arbitrary transform fibers as all W_* roles in the recursive batches. Equation (2.6) establishes the required full linear-map identity.

3 Compiling a Fourier transform on its own coordinates

We next express a Fourier matrix using shallow layers on exactly its given coordinates. Preserving the width is essential: later, the coordinate sets will be tensor factors, so padding them separately would multiply the total dimension. We first factor the Fourier matrix into diagonal and triangular Toeplitz matrices. We then implement those Toeplitz factors on their own coordinates, using the exact-width mechanism of [12, Section 3] with an effective choice of workspace. Finally, we compile every two-coordinate operation to the particular matrix C used above.

A *shear* adds a scalar multiple of one coordinate to a distinct coordinate and leaves all other coordinates unchanged. A *shear layer* consists of shears on disjoint pairs. A *monomial layer* is an invertible monomial matrix. We also use *mixed rounds*: on disjoint coordinate regions, such a round performs monomial layers or shear layers. It splits into a monomial layer followed by a shear layer. This convention lets independent recursive calls run in parallel.

Proposition 3.1 (Uniform local compiler). *Let $r \geq 1$ and let u be any primitive r th root of unity. The matrix $F_r(u) = (u^{ij})_{0 \leq i,j < r}$ has a word on exactly r coordinates with $O(1 + \log^4 r)$ slots, each of which is either*

1. *a monomial layer, or*
2. *disjoint copies of C on ordered pairs, with identity on every remaining singleton.*

There is one fixed repeating pattern of slot types into which all these words can be padded. For any finite collection of widths at most R , the words can therefore be synchronized in $O(1 + \log^4 R)$ slots.

Their schedules and scalar coefficients can be prepared in time $r^{O(1)}$, with an absolute exponent, counting both exact scalar arithmetic and integer bookkeeping. Besides u and the fixed constant i , this preparation needs only power-of-two roots of orders at most $8r$. Coefficients are specified by shared arithmetic programs over these roots and rational numbers. Every division is by a proved nonzero scalar; every structural choice uses integers alone. Neither a test for zero of a complex number nor conjugation of input data is required.

3.1 Reducing Fourier matrices to triangular Toeplitz matrices

The Fourier matrix evaluates a polynomial at the powers of u . Writing that polynomial in the Newton basis makes evaluation triangular. Symmetry then removes the need to compile the change of basis separately.

Lemma 3.2 (Newton reduction). *For every primitive r th root u , with $r \geq 1$, there is an invertible lower triangular matrix N such that*

$$F_r(u) = ND_N^{-1}N^\top, \quad D_N = \text{diag}(N_{00}, \dots, N_{r-1, r-1}).$$

The matrix N is a product of two invertible diagonal matrices and one invertible lower triangular Toeplitz matrix of width r . All these factors can be prepared in polynomial time as rational expressions in u , with every denominator nonzero.

Proof. For $r = 1$ take $N = D_N = T = (1)$. Otherwise let the k th column of P be the coefficient vector of the monic Newton polynomial $\prod_{j=0}^{k-1}(Z - u^j)$, for $0 \leq k < r$. Then P is unit upper triangular and $N = F_r(u)P$ is lower triangular. For $i \geq k$,

$$N_{ik} = \prod_{j=0}^{k-1} (u^i - u^j) = (-1)^k u^{k(k-1)/2} \frac{H_i}{H_{i-k}}, \quad H_j = \prod_{s=1}^j (1 - u^s), \quad H_0 = 1. \quad (3.1)$$

The required H_j , $0 \leq j < r$, are nonzero because u is primitive. Consequently

$$N = \text{diag}(H_i) T \text{diag}((-1)^k u^{k(k-1)/2}), \quad T_{ik} = \begin{cases} H_{i-k}^{-1}, & i \geq k, \\ 0, & i < k. \end{cases}$$

Here T is invertible lower triangular Toeplitz, with diagonal one and coefficient sequence $h_j = H_j^{-1}$.

Let $D_N = \text{diag}(N_{00}, \dots, N_{r-1, r-1})$ and $L = ND_N^{-1}$. The identity $F_r(u) = LD_N P^{-1}$ is a unit-lower/diagonal/unit-upper factorization. Such a factorization is unique: comparing two makes a unit lower triangular matrix equal an upper triangular matrix, forcing both to be identity, and then identifies their diagonal factors. Since $F_r(u) = F_r(u)^\top$, transposition gives another such factorization. Uniqueness yields $P^{-1} = L^\top$, hence

$$F_r(u) = ND_N^{-1}N^\top. \quad (3.2)$$

Computing the powers of u , the H_j , and all diagonal entries has polynomial cost. Every division above is by a product of nonzero H_j and powers of u . In particular, the argument applies to composite r and to every primitive root of order r . $\square$

Thus it suffices to compile an invertible triangular Toeplitz matrix on its own coordinates. Its transpose will require the same number of layers: transposition reverses the factor order and preserves monomial and shear layers. The next two subsections establish this Toeplitz compiler, including the workspace needed by its convolutions.

3.2 Convolution and arbitrary borrowed coordinates

We use linear arithmetic directed acyclic graphs (DAGs) in which each gate forms a linear combination of at most two sources or earlier gate values. Each output is one designated source or gate read, or zero; any output sum or scaling is included among the gates. Fan-out counts these designated output uses as well as uses by later gates.

The elementary convolution circuits will supply the local operations. A convolution of variable data with a fixed vector, whose operand lengths have sum v , has a linear arithmetic DAG with

$$O(v \log(2 + v)) \text{ gates}, \quad O(\log(2 + v)) \text{ depth}, \quad (3.3)$$

and absolute bounded fan-out, counting designated output uses. Gates have at most two predecessors. Reversals and truncations preserve these bounds.

Indeed, pad to the least power of two L at least the full product length. At L th roots of unity, convolution is pointwise multiplication. The transform of the fixed vector supplies scalar coefficients. The binary Fourier recursion separates even and odd coefficients and combines transformed entries a, b into $a + \omega b, a - \omega b$. Each entry is used a bounded number of times at the next level. The inverse uses inverse roots and multiplication by L^{-1} , as follows from the geometric sum of the roots. This proves (3.3), including all multiplications by fixed scalars. Literal zero inputs need no variable registers. Selections have no repeated output indices; a fixed number of these circuits, their compositions, and their sums still have absolute bounded fan-out. Their topology can be printed before any scalar coefficients are evaluated, retaining even gates with a coefficient that might happen to vanish.

The DAG may have extra wires. The following identity implements its effect using coordinates that already carry other data. Its first replay has the compute–copy–reverse pattern of Bennett’s reversible simulation [4]. Restoring storage with arbitrary initial contents is also the principle of clean transparent computation in catalytic space [6, Section 3]. The source-suppressed second replay below proves the exact cancellation and layer bounds needed here.

Lemma 3.3 (Replay with arbitrary borrowed coordinates). *Suppose $M : \mathbb{C}^e \rightarrow \mathbb{C}^a$ has a linear DAG with g gates, depth H , fan-in at most two, and fan-out at most δ , including output uses. Each output is a source or gate read, or zero. On disjoint coordinate sets x, y, z of sizes e, a, g , respectively, a word of $O((\delta + 1)(H + 1))$ shear layers implements*

$$(x, y, z) \mapsto (x, y + Mx, z).$$

It uses at most $8g + 2a$ shears. The initial contents of z are arbitrary, including values correlated with x, y , or other data.

Proof. Assign one coordinate z_v to each gate, in topological order. Instead of assigning the gate value, add its prescribed linear combination of predecessors into z_v . This takes at most two shears. The resulting forward sweep has the form

$$z \mapsto Uz + Tx,$$

where U is unit lower triangular. Write the output reads as $Fz + Dx$, allowing outputs that read a source directly; each nonzero output is one designated read. Starting with zero gate coordinates computes the original DAG, so $FT + D = M$.

Run the forward sweep, add its output reads into y , and undo the sweep. This restores z and adds $Mx + FUz$ to y . Next run the same sweep with all direct-source edges omitted, omit Dx at the readout, subtract the resulting output FUz , and undo the sweep again. The net effect is the stated map. The target coordinates are never predecessors in either sweep. The calculation is an identity in independent variables x, y, z , so it remains valid for correlated initial contents.

To arrange a sweep into layers, assign gates their longest-path depth levels. Within one level, destinations are new gate coordinates and predecessors are earlier gate coordinates or sources. The undirected multigraph of shear edges has maximum degree $\Delta \leq \max\{2, \delta\}$. Greedy edge coloring

uses at most $2\Delta - 1$ colors, since each edge meets at most $2\Delta - 2$ others. The operations in one color have disjoint pairs. Reordering by color is valid because no destination is a predecessor at the same level. The output additions have the same property: their sources have bounded output fan-out and each target has one read. Reverse sweeps use the reversed layers and negated coefficients. Four sweeps and two output rounds give the depth bound and the count $8g + 2a$. $\square$

3.3 Toeplitz layers and a measured workspace rule

The matrix representation in the next proof is the displacement-rank construction of Kailath, Kung, and Morf [10, Lemmas 1–2]: low-rank shift differences reconstruct a matrix through Toeplitz convolution factors. We give the rectangular-chunk formulas and then fit their computation into the available coordinates.

Lemma 3.4 (Effective exact-width Toeplitz layers). *An invertible lower triangular Toeplitz matrix of width r has a word on exactly r coordinates in $O(1 + \log^4 r)$ mixed rounds. A fixed algorithm prepares the word and its coefficients in $r^{O(1)}$ time from the Toeplitz entries and power-of-two roots of orders at most $8r$. The topology depends only on r ; all divisions by Toeplitz parameters are justified by the nonzero diagonal entry.*

Proof. We split the transform into two smaller diagonal blocks and a cross update. For each rectangular part of that update, we will construct a convolution DAG and fit all its gate coordinates inside the parent block. We then count the resulting layers and the preparation work.

Write $T_v = (h_{i-j})_{0 \leq j \leq i < v}$, with zero entries above the diagonal and $h_0 \neq 0$. The same initial sequence h specifies all smaller blocks used below. Its reciprocal-series coefficients are computed by

$$g_0 = h_0^{-1}, \quad g_k = -h_0^{-1} \sum_{j=1}^k h_j g_{k-j} \quad (1 \leq k < r). \quad (3.4)$$

They give the inverse of every lower triangular Toeplitz block.

For $v > 1$, put $p = \lfloor v/2 \rfloor$ and write

$$T_v = \begin{pmatrix} T_p & 0 \\ B & T_{v-p} \end{pmatrix}.$$

The factorization

$$T_v = \begin{pmatrix} I & 0 \\ E & I \end{pmatrix} \text{diag}(T_p, T_{v-p}), \quad E = BT_p^{-1},$$

first performs the two diagonal transforms in parallel, then adds to the second block from the transformed first block. In the original indices of this cross block,

$$E_{ij} = \sum_{\nu=j}^{p-1} h_{i-\nu} g_{\nu-j}, \quad p \leq i < v, \quad 0 \leq j < p. \quad (3.5)$$

Partition both intervals into consecutive chunks. For a target chunk of size a and source chunk of size e , let M be the corresponding $a \times e$ submatrix of E . Let J_a, J_e denote the lower shifts. Away from the first local row and column, (3.5) telescopes to

$$E_{ij} - E_{i-1,j-1} = -h_{i-p} g_{p-j}. \quad (3.6)$$

Thus, writing $\mathcal{D} = M - J_a M J_e^\top$, we can form

$$\mathcal{D} = \sum_{\nu=1}^3 v^{(\nu)} (w^{(\nu)})^\top \quad (3.7)$$

without any rank test. Specifically, extend the outer product on the right of (3.6) to the entire chunk and subtract it from $\mathcal{D}$, obtaining R . If e_0 is the first unit vector, the two remaining outer products are $e_0 R_{0,*}$ and $(R_{*,0} - R_{00} e_0) e_0^\top$. They correct its first row and first column. Keep all three terms, even when some coefficients vanish.

Iterating $M = \mathcal{D} + J_a M J_e^\top$ gives, by nilpotence, the finite reconstruction

$$M = \sum_{l \geq 0} J_a^l \mathcal{D} (J_e^\top)^l. \quad (3.8)$$

For one outer product $vw^\top$, its contribution has entries $\sum_{l \geq 0} v_{i-l} w_{j-l}$, with zero extension. Apply it to x by

$$b_l = \sum_{j=l}^{e-1} w_{j-l} x_j \quad (0 \leq l < e), \quad c_i = \sum_{l=0}^{e-1} v_{i-l} b_l \quad (0 \leq i < a). \quad (3.9)$$

If $x_j^{\text{rev}} = x_{e-1-j}$, the first map selects $b_l = (w * x^{\text{rev}})_{e-1-l}$ from an ordinary convolution. The second map is a truncated convolution. Using three branches and summing their outputs gives a specific DAG with absolute constants K_0, K_1, δ_0 such that

$$g \leq K_0(a+e) \log_2(2+a+e), \quad H \leq K_1 \log_2(2+a+e), \quad \delta \leq \delta_0. \quad (3.10)$$

These include output uses: every source is used in only three branches, the branches retain bounded fan-out, and their sum uses a bounded number of gates per output.

To fit this DAG inside the parent block, reserve the $a+e$ target and source coordinates and one borrowed coordinate for each of its g gates. All g coordinates stay assigned to their gates throughout the replay. Here is an effective choice of chunk size that does not require knowing K_0 . For every integer $b \in \{1, \dots, v\}$, partition each interval into chunks of size b , except for its possible last chunk. Print the DAG skeleton for every resulting pair and count its gates. Declare b admissible precisely when, for every pair,

$$g + a + e \leq v. \quad (3.11)$$

Choose the largest admissible b . This is an integer test on printed graphs, independent of the coefficients, made before any data are transformed. If no b is admissible, replace the entire recursive decomposition of T_v by a direct word: process rows in decreasing order, multiply the current coordinate by h_0 , and add h_{i-j} times each lower-indexed coordinate $j < i$. The lower coordinates still have their original values, so this computes exactly T_v . Width one is a scaling.

To prove the depth bound, fix for analysis a sufficiently large absolute D , for example $D \geq 16K_0 + 16$. For all sufficiently large v ,

$$b_0 = \left\lfloor \frac{v}{D \log_2 v} \right\rfloor \geq \frac{v}{2D \log_2 v} \geq 1$$

passes (3.11). Indeed $a+e \leq 2v/(D \log_2 v) \leq v/8$, while (3.10) gives $g \leq 4K_0 v/D \leq v/4$. Consequently the direct fallback occurs only for bounded v , and the chosen size $b \geq b_0$ leaves $O(\log v)$ chunks in each interval.

For a selected pair, borrow g coordinates outside its source and target chunks but inside these same v coordinates; (3.11) guarantees their availability. Lemma 3.3 implements its update in $O(1 + \log v)$ shear layers. Process the $O(\log^2 v)$ pairs serially. Every borrowed coordinate is restored before the next pair, so it may carry a source value, an already updated target value, or any other current data. The complete cross update takes $O(1 + \log^3 v)$ layers. Child calls use disjoint blocks, hence the mixed-round depth satisfies

$$d(v) \leq \max\{d(\lfloor v/2 \rfloor), d(\lceil v/2 \rceil)\} + O(1 + \log^3 v).$$

Summing along a balanced recursion path proves $d(r) = O(1 + \log^4 r)$.

We finally account for preparation. Formula (3.4), the entries (3.5), and the three explicit outer products use polynomially many scalar operations. Each node of the recursion tests at most v sizes and at most v^2 chunk pairs per size; every printed graph has $O(v \log(2 + v))$ gates. There are fewer than $2r$ nodes. Even this deliberately redundant search has polynomial integer cost. Gate-level assignments, bounded-degree edge colorings, and the choice of the first available borrowed coordinates can all be performed by finite scans of these lists. Only the chosen graphs need their scalar coefficients evaluated. Fixed-vector transforms and all other coefficient computations have polynomial cost as well. Store computed scalars and references to them as an arithmetic DAG; there is no expansion into expression trees.

The first convolution in (3.9) has operand lengths e, e , and the second has lengths a, e . Their padded Fourier lengths are powers of two smaller than $4r$. Thus a single primitive root of power-of-two order $L_r = 2^{\lceil \log_2(4r) \rceil} < 8r$ supplies every needed root by taking powers. These powers and their inverses are also prepared with polynomially many operations. Integer indices in the entire compiler have $O(\log(2 + r))$ bits. This proves the claimed preparation bound, with no need to determine the finite threshold in the depth argument. $\square$

3.4 Fourier layers and a fixed word in the kernel

Lemma 3.5 (Fourier layers at every primitive root). *For every primitive r th root u , the matrix $F_r(u)$ has a polynomially preparable exact-width word in $O(1 + \log^4 r)$ monomial and shear layers. Its coefficients are rational expressions in u and the power-of-two roots from Lemma 3.4, with all denominators nonzero.*

Proof. Apply Lemma 3.4 to the Toeplitz factor of N in Lemma 3.2. The two diagonal factors of N are monomial layers. Equation (3.2) then gives the Fourier word by composing the words for N^T , D_N^{-1} , and N . Transposition preserves the layer types and reverses the factor order. Split mixed rounds into monomial and shear layers. There are $O(1 + \log^4 r)$ layers in total, and all preparation bounds and nonzero-denominator guarantees follow from the two lemmas. $\square$

We complete the proof of Proposition 3.1. Recall $C = \begin{pmatrix} a & b \\ b & a \end{pmatrix}$, $a = (1 + i)/2$, $b = (1 - i)/2$. Set

$$H' = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad S = \text{diag}(1, i).$$

Direct multiplication gives $H' = a^{-1}SCS$ and

$$K := H' \text{diag}(2, 1) H' \text{diag}(-3, 1) H' = \begin{pmatrix} -8 & -10 \\ 0 & -6 \end{pmatrix}. \quad (3.12)$$

Thus every upper shear with nonzero coefficient t has a word with exactly three forward copies of C and invertible diagonal factors:

$$\begin{pmatrix} 1 & t \\ 0 & 1 \end{pmatrix} = D_t \operatorname{diag}(-1/8, -1/6) K D_t^{-1}, \quad D_t = \operatorname{diag}(4t/5, 1). \quad (3.13)$$

An update $y \leftarrow y + tx$ is this upper shear on the ordered pair (y, x) . The order of a pair is part of the discrete schedule.

A general coefficient μ may vanish, but no test is necessary. Prepare

$$\kappa = 1 + \mu\bar{\mu}, \quad t_1 = \kappa, \quad t_2 = \mu - \kappa. \quad (3.14)$$

Both t_1, t_2 are nonzero, since $1 + |\mu|^2 > |\mu|$. The two corresponding shears compose to the desired shear, including when $\mu = 0$. Equations (3.12)–(3.14) therefore give the same six- C slot pattern for every coefficient. On disjoint pairs these words run in parallel; every unused coordinate is an identity singleton in every C slot.

The conjugate in (3.14) is a prepared scalar, not an operation on variable data. Every coefficient μ from Lemma 3.5 is given by an arithmetic DAG over rational numbers and the specified roots. Evaluate a second copy of that DAG with each root replaced by its inverse. Rational constants are real, so the result is precisely $\bar{\mu}$. The second evaluation has the same size, and its denominators are conjugates of nonzero denominators. This argument also includes the fixed root i , whose conjugate is $-i$. Shared subexpressions remain shared, so preparing all conjugates increases polynomial preparation cost by at most a constant factor before the constant-size replacements.

Finally, place each layer from Lemma 3.5 into a monomial slot followed by a shear slot, using identity for the other slot type, and replace the shear slot by this fixed pattern. Combine adjacent monomial slots if desired. An idle round uses identity monomials and only singleton identities in its C slots. All words now repeat a common constant-size pattern. Padding the shorter words with idle rounds synchronizes any finite collection without increasing the $O(1 + \log^4 R)$ bound. This proves Proposition 3.1.

4 Synchronizing tensor factors in linear array work

This section turns the tensor-power algorithm and the local compiler into a transform on a product of small coordinate sets. The indexing estimate is as important as the scalar estimate: scanning all the axes anew at each array entry would lose the saving.

4.1 A linear-time traversal rule

For radices $r_1, \dots, r_\ell \geq 2$, let $R = \prod_j r_j$. We order coordinates by mixed-radix tuples, with the first digit most significant. A recursive traversal of the digit tuples visits fewer than $2R$ nodes, since

$$\sum_{j=0}^{\ell} \prod_{i=1}^j r_i \leq R \sum_{j=0}^{\ell} 2^{j-\ell} < 2R. \quad (4.1)$$

Any state updated in a fixed number of operations per prefix extension can therefore be computed for all coordinates in $O(R)$ time. A depth-first traversal uses a stack; it does not copy the whole prefix at every extension.

For example, a tensor monomial maps each local digit a_j to $\pi_j(a_j)$ and multiplies by a scalar $c_j(a_j)$. Extend the original and permuted mixed-radix addresses by the usual multiply-and-add rule, and extend a running coefficient by multiplication by $c_j(a_j)$. At a leaf, move and scale one entry. Equation (4.1) gives $O(R)$ operations, including the scalar products.

4.2 Packing sectors

A layer of disjoint copies of C on an axis partitions its coordinates into ordered pairs and unused singletons. Choose any order of these blocks. For a selected block on axis j , let

$$q_j \in \{1, 2\}, \quad p_j = \text{the number of coordinates in preceding blocks},$$

and let $t_j \in \{0, \dots, q_j - 1\}$ denote position within that block. These local tables are part of the compiled layer. The actual axis digit, denoted a_j , need not be consecutive in the original coordinate order.

A *sector* chooses one block on each axis. It has width $Q = \prod_j q_j = 2^k$, where k is the number of chosen pairs. Order sectors lexicographically by their block choices, and each sector internally by its t_j digits. Put $\tau_j = \prod_{i>j} r_i$.

Lemma 4.1. *The start of the selected sector in the packed array is*

$$S = \sum_{j=1}^{\ell} \left(\prod_{i<j} q_i \right) p_j \tau_j. \quad (4.2)$$

The packing permutation, its inverse, and the list of sector starts, widths, and pair counts can all be generated in $O(R)$ operations after the local tables and suffix products have been prepared.

Proof. Classify the earlier sectors by the first axis j at which their block choice differs. The preceding chosen blocks contribute width $\prod_{i<j} q_i$, the earlier blocks at axis j have total width p_j , and all later axes have total width τ_j . These disjoint classes give (4.2).

During digit traversal maintain four states: a sector start S , a prefix width Q , a within-sector offset o , and an original address I . Initially they are 0, 1, 0, 0. On extending by the actual local digit a_j , whose table supplies p_j, q_j, t_j , update

$$\begin{aligned} S &\leftarrow S + Q p_j \tau_j, & Q &\leftarrow Q q_j, \\ o &\leftarrow o q_j + t_j, & I &\leftarrow I r_j + a_j. \end{aligned} \quad (4.3)$$

At the leaf, the packed address is $S + o$. This proves the cost of packing and unpacking by (4.1).

To enumerate calls, traverse block choices instead of digits, maintaining S, Q , and the number of pairs. If axis j has b_j blocks, then $b_j \leq r_j$. Thus the number of block prefixes at every depth is bounded by the number of digit prefixes at that depth, and their total is also $O(R)$. This argument includes axes with only one block. Each leaf supplies one contiguous sector, with no additional scan through its axes. $\square$

4.3 The synchronized transform

Proposition 4.2. *Let $\ell \geq 1$ and $r_1, \dots, r_\ell \geq 2$ be integers, and suppose the required specified roots are given. Put $R = \prod_j r_j$ and $r_{\max} = \max_j r_j$. The map $\bigotimes_j F_{r_j}$ is computable in*

$$O\left(R(\ell + 1)^\theta (1 + \log r_{\max})^4\right) + \text{poly}(\ell, r_{\max}, \log(R + 2)) \quad (4.4)$$

operations, including compilation, scalar preparation, and array work. The polynomial and the implicit constant are absolute.

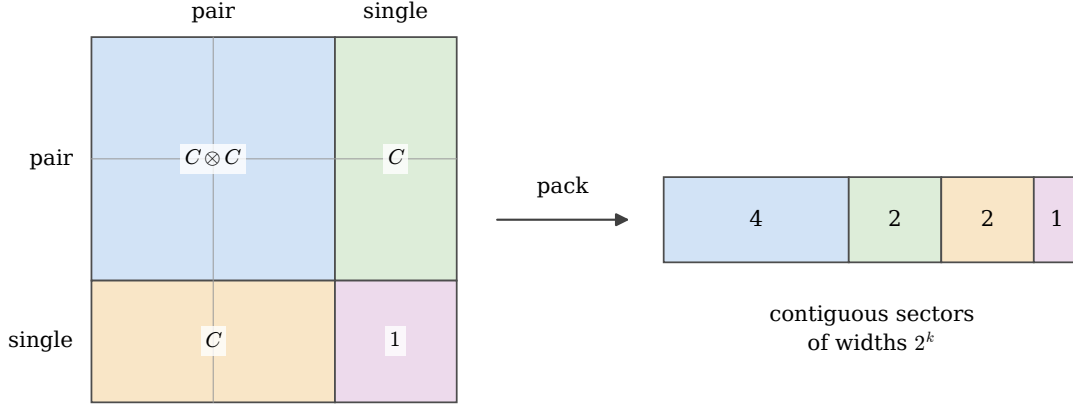


Figure 2: Two axes, each partitioned into a pair and a singleton. A simultaneous pair layer becomes four independent tensor-power calls. The original pair coordinates can be nonconsecutive; the local tables and (4.3) perform the reordering in linear time.

Proof. Use Proposition 3.1 on every axis. Its words can be refined and padded to a common sequence of $O((1 + \log r_{\max})^4)$ slot types: a monomial slot or a slot of disjoint copies of C and singleton identities. Padding a pair slot means using only singleton blocks on that axis.

At each slot take the tensor product across axes. These slots multiply to the desired tensor transform because

$$\left(\bigotimes_j A_j\right)\left(\bigotimes_j B_j\right) = \bigotimes_j (A_j B_j). \quad (4.5)$$

In particular, a local word's correctness on arbitrary inputs suffices. Another axis may change coordinates during intermediate slots; one does not require it to preserve the local word's borrowed coordinates throughout that word.

Tensor monomials cost $O(R)$ by the prefix traversal. A tensor pair slot decomposes into the sectors of Lemma 4.1. Its width- 2^k sector carries precisely $C^{\otimes k}$. Theorem 2.6 bounds its cost, including invocation and movement, by $O(2^k(k+1)^\theta)$. As $k \leq \ell$ and the sector widths sum to R , their total cost is $O(R(\ell+1)^\theta)$. Packing, unpacking, and discovering even the width-one sectors cost another $O(R)$. This proves the first term of (4.4).

The local compilers use polynomial work in their widths. Suffix products, local block tables, and the other integer preparation use polynomial work in $\ell, r_{\max}, \log(R+2)$. All running scalar products have been charged in the traversal, so the polynomial term describes local preparation only. $\square$

5 Working lengths and the transform at every length

To apply the synchronized transform at a requested length n , we first choose a working length L with $2n \leq L < 4n$. The lower bound permits the chirp convolution below, while the upper bound keeps its array size linear in n . We also need small coprime factors: their widths must be polynomial in $\log n$ so that local compilation has only polylogarithmic cost. A product of initial odd primes supplies these axes, and one power of two fills the remaining gap to $2n$.

5.1 Enough small, pairwise coprime factors

Let $r_1 < r_2 < \dots$ be the odd primes and let $R_\ell = \prod_{j=1}^\ell r_j$, with $R_0 = 1$. The modest estimate below suffices; no prime number theorem is used.

Lemma 5.1. *One has $r_j = O((j+1)^2)$ and*

$$\log R_\ell = \Theta(\ell \log(\ell + 2)).$$

For the largest ℓ with $R_\ell \leq 2n$, as $n \rightarrow \infty$,

$$\ell = \Theta\left(\frac{\log n}{\log \log n}\right), \quad \log \ell = \Theta(\log \log n). \quad (5.1)$$

The primes and this value of ℓ can be found with a number of operations polynomial in $\log(n+2)$, using $O(\log(n+2))$ -bit integers.

Proof. Writing $\pi(t)$ for the number of primes at most t , for integer $u \geq 1$ we have

$$\frac{4^u}{2u+1} \leq \binom{2u}{u} \leq (2u)^{\pi(2u)}. \quad (5.2)$$

The first inequality follows because the central binomial coefficient is largest. In the factorial formula, the exponent of a prime p in $\binom{2u}{u}$ is

$$\sum_{a \geq 1} (\lfloor 2u/p^a \rfloor - 2\lfloor u/p^a \rfloor) \leq \lfloor \log_p(2u) \rfloor.$$

Its prime-power contribution is therefore at most $2u$, proving the second inequality. Taking $u = (j+1)^2$ shows that $\pi(2u) > j+1$ for all sufficiently large j , so the j th odd prime is $O((j+1)^2)$.

The upper bound on $\log R_\ell$ now follows by summation. For the lower bound, $r_j \geq j+2$, so $R_\ell \geq (\ell+2)!/2$. For instance, the last half of these factors alone give a constant times $\ell \log \ell$ in the logarithm. Maximality says

$$R_\ell \leq 2n < R_\ell r_{\ell+1}.$$

The extra logarithm is $O(\log(\ell+2))$, giving $\log n = \Theta(\ell \log \ell)$ and hence (5.1).

Primes up to the first unused prime are bounded by a fixed polynomial in $\ell+1 = O(\log(n+2))$. Testing potential divisors of each candidate is already polynomial work in $\log(n+2)$. Maintain the product and stop at the first exceeding $2n$; the product at stopping is at most $2n$ times a polynomial in $\log(n+2)$. All these integers have $O(\log(n+2))$ bits. $\square$

Choose this ℓ , and let $e \geq 0$ be the least integer for which

$$L = R_\ell 2^e \geq 2n. \quad (5.3)$$

Then

$$2n \leq L < 4n, \quad 2^e < 2r_{\ell+1}, \quad e = O(\log(\ell+2)). \quad (5.4)$$

Bounded lengths can be treated directly, so subsequent asymptotic estimates may assume $\ell \geq 1$.

5.2 CRT permutations and the working transform

We use Good's coprime tensor factorization [9, Section 12], with its index permutations included in the cost.

For pairwise coprime factors q of L , here the odd primes and the factor 2^e when nontrivial, choose d_q with

$$(L/q)d_q \equiv 1 \pmod{q}, \quad E_q = (L/q)d_q.$$

The integers E_q are orthogonal idempotents modulo L . The Chinese remainder indexing

$$(j_q)_q \mapsto \sum_q E_q j_q \pmod{L}$$

is a bijection, and for similarly indexed k ,

$$\zeta_L^{jk} = \prod_q \zeta_q^{d_q j_q k_q}. \quad (5.5)$$

Thus, after permutations of the input and output indices, F_L is the tensor product of the F_q . The local output permutation multiplies a digit by the unit d_q modulo q .

These permutations take $O(L)$ array operations. Prepare the local tables $E_q a \bmod L$ and $d_q a \bmod q$, then enumerate digit tuples with an accumulated sum and mixed-radix address. The prefix bound (4.1) applies; no division by every factor is performed separately for each array entry. Computing the inverses d_q and local tables is polynomial in the logarithm of n .

Apply Proposition 4.2 on the odd-prime axes, with each digit of the power-of-two axis fixed in turn. Then apply the usual radix-two FFT on the fibers of that remaining axis. The former costs

$$O\left(L(\ell+1)^\theta(1+\log(\ell+2))^4\right) + \text{poly}(\log(n+2)),$$

because $r_\ell = O((\ell+1)^2)$; the latter costs $O(L(1+e))$. Local tables are shared among the fibers. The standard binary recursion uses $O(q \log(q+1))$ operations at length $q = 2^e$, with $O(q)$ twiddle preparation and ordinary linear-time stage indexing. Here q is only polynomial in $\ell+1$.

Consequently both F_L and its inverse have total cost

$$O\left(L[(\ell+1)^\theta(1+\log(\ell+2))^4 + 1 + e]\right) + \text{poly}(\log(n+2)) + O(1). \quad (5.6)$$

For the inverse, the geometric root sum gives $F_L^2 = LJ$, where J negates indices modulo L . Thus $F_L^{-1} = L^{-1}JF_L$, with only linear additional work. The absolute term includes the fixed network construction.

5.3 Chirp convolution and preparation

The reduction is Bluestein's chirp convolution [5], in the weighted-convolution form of [14, Section II]. We charge preparation of the fixed convolution operand explicitly.

Put $\eta = \zeta_{2n}$. For $0 \leq j, k < n$,

$$\zeta_n^{jk} = \eta^{k^2} \eta^{-(k-j)^2} \eta^{j^2}. \quad (5.7)$$

Prepare two length- L vectors. The input-dependent vector has value $\eta^{j^2} x_j$ at $0 \leq j < n$ and zero elsewhere. The fixed vector has value η^{-u^2} at the residues represented by $-(n-1) \leq u \leq n-1$, and zero elsewhere. These residues are distinct by $L \geq 2n$. Cyclic convolution of the vectors therefore gives, at each $0 \leq k < n$, the sum in (5.7) before multiplication by η^{k^2} .

The convolution is obtained by applying F_L to both vectors, multiplying pointwise, and applying F_L^{-1} . The transform of the fixed vector is part of scalar preparation and is explicitly charged as one full transform. Its pointwise use multiplies the variable data only by prepared scalars. Chirp tables and the zero-padded vectors take $O(L) = O(n)$ work: successive ratios of the chirps form a geometric progression, so no separate long exponentiation is needed at every entry. There are only three full transforms.

We now specify the single root promised in Theorem 1.1. With L as in (5.3), set

$$D_* = (2n)L 2^{\lceil \log_2(16L) \rceil}. \quad (5.8)$$

The chirp root has order $2n$, and the working factor roots have orders dividing L . Proposition 3.1 needs power-of-two roots of orders at most $8r$ for a local width $r \leq L$; each such order divides the last factor in (5.8). The fixed kernel uses only i and rational constants. Thus every required initial root is an integer power of the supplied, specified ζ_{D_*} . Moreover,

$$D_* < 64nL^2 < 1024n^3. \quad (5.9)$$

Binary powering extracts each local initial root in $O(\log(n+2))$ field operations. There are polynomially many local constants and instructions in $\log(n+2)$, so all local extraction and compilation remain polynomial in that logarithm.

The local scalar programs are arithmetic DAGs with shared subexpressions. As in Proposition 3.1, a coefficient's conjugate is prepared by evaluating the same DAG with unchanged rational literals and inverse Fourier roots, including $i^{-1} = -i$. Thus no conjugation of variable input data or zero decision about complex coefficients is required. Full-array scalar products and the transformed convolution kernel have already been charged in (5.6); only the *local* preparation is polynomial in $\log(n+2)$.

5.4 The final estimates

Proof of Theorem 1.1 and Corollary 1.2. All maps in the construction are exact and deterministic. The local word compiler and the fixed finite network have specified finite construction procedures. On the input data the algorithm is linear, including the chirp reduction, since the second convolution operand is fixed.

By (5.4), $e = O(\log(\ell+2))$ and $L = \Theta(n)$. Because $\theta > 0$, equation (5.6) and the constant number of transforms give

$$T(n) = O(n\ell^\theta(\log \ell)^4) + \text{poly}(\log(n+2)) + O(1).$$

Every fixed power of $\log(n+2)$ is eventually negligible compared with $n\ell^\theta$. Substituting (5.1) proves (1.2).

For an exact bound on the exponent, direct integer arithmetic in (1.1) gives

$$\varepsilon := \frac{\Delta}{mW_*} = \frac{1717850673}{590295810358705651712} > \frac{28}{10^{13}}.$$

Since $\log m < 14$, for $\eta_0 = 2/10^{13}$ one has $\eta_0 \log m < \varepsilon$. Using $\exp(-t) \geq 1 - t$,

$$\lambda = m(1 - \varepsilon) < m \exp(-\eta_0 \log m) = m^{1-2 \cdot 10^{-13}}.$$

Hence $\theta < 1 - 2 \cdot 10^{-13}$. The fixed power of $\log \log n$ in (1.2) is smaller than $(\log n)^{10^{-13}}$ for all sufficiently large n . This proves Corollary 1.2.

Finally, all lengths, root orders, products of indices, and addresses are polynomially bounded in n with absolute constants; (5.9) treats the largest root order explicitly. Fixed network tables contribute absolute constants, and local tables have size polynomial in $\log(n+2)$. Array movement and traversal were counted in Theorem 2.6 and Lemma 4.1. Therefore a fixed number of $O(\log(n+2))$ -bit words per integer suffices throughout, as required by the model. $\square$

5.5 Two-input convolution and polynomial multiplication

The chirp reduction above convolves input data with a fixed vector and therefore remains linear. For two variable operands, the same Fourier identity gives Corollary 1.3, with one product of two input-dependent values at each transform coordinate.

Proof of Corollary 1.3. Set $M = 2n - 1$ and zero-pad a and b to vectors $\tilde{a}, \tilde{b}$ of length M . Every sum of two indices in their supports lies between zero and $2n - 2 = M - 1$, so the length- M cyclic convolution has no wraparound and is exactly $(c_0, \dots, c_{M-1})$. With the positive-exponent convention for F_M , its convolution identity is

$$(F_M c)_j = \sum_{r,s=0}^{n-1} a_r b_s \zeta_M^{j(r+s)} = (F_M \tilde{a})_j (F_M \tilde{b})_j \quad (0 \leq j < M).$$

Compute the two forward transforms and their M pointwise products, obtaining a vector $p = F_M c$. If J negates indices modulo M , the geometric root sum gives $F_M^2 = MJ$ and therefore

$$c = F_M^{-1} p = M^{-1} J F_M p.$$

Thus one more forward transform, index reversal, and scaling give the answer.

Apply Theorem 1.1 at length M for each of the three transforms, charging its scalar preparation, schedule construction, and index work each time. The single specified root supplied for that length can be reused in all three calls. Its explicitly computable order satisfies $D_* < 1024M^3 < 8192n^3$; equivalently, the model's root-of-unity operation returns this one root. Padding, pointwise products, reversal, and output scaling take $O(M)$ field and address operations. To prepare M^{-1} , form the complex integer M from 1 by binary doubling and addition in $O(\log(M+2))$ field operations, then divide 1 by it; the denominator is nonzero because $M \geq 1$. All added indices and addresses fit a fixed number of $O(\log(n+2))$ -bit words.

If $T(M)$ denotes the charged running time in Theorem 1.1, the total is $3T(M) + O(M + \log(M+2))$. Since $M = \Theta(n)$, this gives the first bound, and Corollary 1.2 gives the eventual decimal power bound. The same formulas apply when $n = 1$; a finite initial range may also be handled directly. $\square$

A Effective synthesis and uniform compilation

This appendix gives a general route from a finite tensor saving to a uniform Fourier algorithm. Its purpose is to separate two questions: how to obtain usable coefficients from an existence theorem over $\mathbb{C}$, and how to use effective small circuits when their compilation time has no useful bound. The explicit network in the main argument gives the stronger quantitative conclusion without this search. We first replace a complex certificate by identities in a fixed finite-dimensional rational algebra. Matrix representations of that algebra then give ordinary scalar circuits without choosing a complex embedding. We print those circuits together with their scalar preparation programs, and finally limit the printer's running time to select a sufficiently small compiled radix for each input length.

The certificate used below consists of two finite words for the same invertible nonmonomial matrix $A \in \text{GL}_q(\mathbb{C})$, with integers $q, b \geq 2$. The first word acts on exactly q^b coordinates as $A^{\otimes b}$ and uses

$$g < bq^{b-1} \quad (\text{A.1})$$

forward calls to A , interspersed with invertible monomials. A call acts on an ordered q -tuple and fixes every other coordinate. The second word acts on q coordinates as $U = I_q + E_{12}$, where E_{12} is a matrix unit, using a positive number of forward calls to that same A and invertible monomials.

The companion paper [12, Theorem 8.3 and Lemma 2.2] obtains such a pair from a general finite-win theorem and positive generation by an invertible nonmonomial matrix. The following lemma supplies the particular certificate needed here directly from our network. Thus the search-termination proof is also available locally. The extraction and printer constructions that follow use only the two word identities, so they retain their general certificate interface.

Lemma A.1 (A local certificate for the symbolic search). *For the invertible nonmonomial matrix C in (2.1), there are integers $b \geq 2$ and $g \geq 0$ and a word on exactly 2^b coordinates which implements $C^{\otimes b}$ using $g < b2^{b-1}$ forward calls to C and invertible monomials. On two coordinates, $I_2 + E_{12}$ has a word with exactly three forward calls to the same C and invertible diagonal factors.*

Proof. Use Proposition 2.4 and the padding (2.10) on $W_* = 2^{71}$ arrays of length 2^{mf} . The $S = W_*m - \Delta$ directional steps require exactly $Sf2^{mf-1}$ forward C -calls when their tensor factors are applied separately: one step has $2^{(m-1)f}$ fibers, each requiring $f2^{f-1}$ calls. Inverse steps add only translations. Each scalar row of (2.3) is a composition of elementary shears between existing, distinct roles; its source coordinates are unchanged within that row. The coefficients are nonzero rational numbers. Equation (3.13) therefore bounds the pointwise work by $H2^{mf}$ forward calls for a fixed integer H independent of f . All remaining operations are invertible monomials. Every role carries arbitrary data, so this word uses exactly W_*2^{mf} coordinates and acts as $I_{W_*} \otimes C^{\otimes mf}$. Address permutations here are monomial factors on these coordinates; no implementation buffer is added to the word's width.

Identify the role index with 71 binary axes and apply C along each of them. This costs $71W_*2^{mf-1}$ calls and gives $C^{\otimes b}$ with $b = mf + 71$. Its actual number g of forward calls therefore satisfies

$$g \leq (Sf + 71W_* + 2H)2^{mf-1} = b2^{b-1} - (\Delta f - 2H)2^{mf-1} < b2^{b-1}$$

for any integer $f > 2H/\Delta$. Finally, $\det C = i$ and both scalar coefficients in (2.1) are nonzero, so C is invertible and nonmonomial. Equation (3.13) with $t = 1$ supplies the claimed three-call word for $I_2 + E_{12}$. Any embedded call on an ordered pair is a permutation conjugate of $C \oplus I_{2b-2}$; the permutations can be absorbed into adjacent monomial factors. $\square$

A.1 Obtaining finite rational algebra data

The extraction uses the triangular monic substitution from the standard proof of Noether normalization [16, Tags 051M, 051N, and 00OX] and the determinant form of Nakayama's lemma [15]. We give the rational algorithm and its termination proof in full.

Lemma A.2 (Effective finite algebra extraction). *Given finitely many polynomials f_i in n variables over $\mathbb{Q}$, a deterministic rational algorithm either certifies that (f_i) is the unit ideal or produces a nonzero finite-dimensional commutative unital $\mathbb{Q}$ -algebra E , with a rational multiplication table, and elements $\xi_1, \dots, \xi_a \in E$ satisfying every $f_i(\xi) = 0$. In particular, a complex solution guarantees the second outcome. Required units can be imposed by adding inverse-witness equations.*

Proof. Keep a finite generating list for the ideal $I = (f_i)$ and an integer m , initially $m = a$. Retain polynomial-combination witnesses for derived members of I ; rational elimination and the monic reductions below can track these witnesses. After invertible polynomial changes of coordinates, maintain, for each $j > m$, a known member of I of the form

$$x_j^{d_j} + \sum_{e < d_j} c_{je}(x_1, \dots, x_{j-1})x_j^e, \quad d_j \geq 1.$$

Thus $S = \mathbb{Q}[x_1, \dots, x_a]/I$ is finite as a module over $R = \mathbb{Q}[x_1, \dots, x_m]$.

First compute the fiber $S/(x_1, \dots, x_m)S$. This is effective: set $x_1 = \dots = x_m = 0$ and let B be the quotient by only the displayed monic relations. As a tower of monic extensions, B has a rational basis consisting of the monomials whose x_j -exponents are less than d_j for $j > m$. Successive monic reduction gives its multiplication table. If $\bar{f}_i$ are the images in B of the generators of I , form the rational span of all $b\bar{f}_i$ with b in this basis. Since the basis spans B , this is exactly the ideal $\sum_i B\bar{f}_i$. Its quotient is the required fiber, so rational linear algebra computes the fiber and its table. A nonzero fiber is the requested answer, after undoing the coordinate changes. If $m = 0$ and the fiber is zero, then $S = 0$, so I is the unit ideal.

Suppose the fiber is zero and $m > 0$. We claim $I \cap R \neq 0$. Otherwise $R \hookrightarrow S$. Localize at $\mathfrak{m} = (x_1, \dots, x_m)$ in R . The resulting finite $R_{\mathfrak{m}}$ -module $S_{\mathfrak{m}}$ is nonzero, since it still contains $R_{\mathfrak{m}}$. The zero fiber would give $S_{\mathfrak{m}} = \mathfrak{m}S_{\mathfrak{m}}$. For a finite generating column v this means $v = Mv$ with all entries of M in the maximal ideal. Multiplication by the adjugate of $I - M$ gives $\det(I - M)v = 0$; its determinant is a unit, a contradiction.

Find a nonzero member $h \in I \cap R$ by enumerating increasing degree cutoffs for monomial multiples of the generators of I . At each cutoff, rational linear algebra tests whether their span has a nonzero element involving only the first m variables. This search terminates by the claim and the definition of an ideal. A nonzero constant certifies $I = (1)$. Otherwise let $d = \deg h \geq 1$ and substitute

$$x_i = y_i + y_m^{(d+1)^i} \quad (1 \leq i < m), \quad x_m = y_m,$$

leaving later variables unchanged. This change is invertible. The weights $1, (d+1), \dots, (d+1)^{m-1}$ distinguish all monomials of total degree at most d by base- $(d+1)$ expansion. Consequently the highest power of y_m in the transformed h has a nonzero rational leading coefficient: it comes from a unique monomial, taking the pure-power term in every substituted factor. Divide by that coefficient to obtain a monic relation in y_m . Earlier triangular relations remain monic in their respective later variables. Rename the variables and decrease m by one. At most a such decreases occur, proving termination. Every step uses finite rational arithmetic. Finally, the equation $uv = 1$ forces u to be a unit in every output algebra, including one with zero divisors. $\square$

Enumerate integer tuples (q, b, g, h) with $q, b \geq 2$, $g \geq 0$, $h \geq 1$ and (A.1). For each tuple form polynomial equations for

$$A^{\otimes b} = M_0 Y M_1 Y \cdots Y M_g, \quad U = N_0 A N_1 A \cdots A N_h, \quad Y = A \oplus I_{q^b - q}. \quad (\text{A.2})$$

All embedded placements are absorbed into adjacent monomials. Include a matrix inverse witness for A . In a monomial slot write a variable permutation factor P and a variable diagonal factor; impose $p_{ij}^2 = p_{ij}$ and row and column sums equal to 1 on P , and give every diagonal entry an inverse witness. These are finite polynomial systems over $\mathbb{Q}$. Apply Lemma A.2 in a fixed enumeration order until one succeeds. Lemma A.1 gives a complex solution for a tuple with $q = 2$ and $h = 3$, so success is guaranteed.

The entries p_{ij} in the resulting E may be nonscalar idempotents. Resolve them one at a time. For a current idempotent $p \neq 0$, replace E by the nonzero algebra pE with unit p and project every stored element by $z \mapsto pz$; if $p = 0$, do nothing. This is a unital homomorphism onto the new algebra and makes the current idempotent equal to its unit. Previously resolved values remain 0 or 1. After finitely many rational table operations all permutation entries are 0 or 1, and their row and column sums give actual permutations. All identities, inverse witnesses, and diagonal units survive. Choose a rational basis of the final nonzero algebra beginning with 1_E . We have thus computed fixed, fully specified data (A.2) over E .

Remark A.3 (Faithful scalar simulation). If $d = \dim_{\mathbb{Q}} E$, then $E \otimes_{\mathbb{Q}} \mathbb{C}$ is the vector space $\mathbb{C}^d$ with the computed multiplication table. The embedding $z \mapsto z1_E$ is injective. Addition is coordinatewise and multiplication by any prepared algebra element is a $d \times d$ complex matrix, so each linear algebra operation costs $O(d^2)$ ordinary scalar operations. When a circuit acts as F_r times the algebra identity, embed its inputs as multiples of 1_E and read the first coordinate of each output. This is a faithful, constant-factor simulation; choosing a complex embedding or extracting algebraic roots is unnecessary. Here d is fixed before the input length is known.

A.2 A printer for increasingly efficient radices

We record why the fixed identities suffice for an effective circuit printer. The tensor amplification of [12, Lemma 2.3] also works over E : its proof uses only the identities and linear operations. Explicitly, put $Q = q^b$, $j = \lfloor k/b \rfloor$ and tensor the first word in (A.2) across j groups. Every call slot decomposes, after reindexing, as

$$(A \oplus I_{Q-q})^{\otimes j} \cong \bigoplus_{r=0}^j (A^{\otimes r})^{\oplus \binom{j}{r}} (Q-q)^{j-r}.$$

Recurse on these smaller tensor orders and use direct calls on the fewer than b remaining axes. Monomial scalings and copies cost linear size at each level. Let $f(k)$ be the size of this recursive construction divided by q^k . The total width of the sectors with r active factors, divided by Q^j , is

$$\frac{\binom{j}{r} (Q-q)^{j-r} q^r}{Q^j} = \binom{j}{r} (q/Q)^r (1 - q/Q)^{j-r}.$$

Thus the normalized cost satisfies

$$f(k) \leq C_0 + g \mathbb{E}f(K), \quad K \sim \text{Bin}(\lfloor k/b \rfloor, q/Q).$$

The distribution weights sectors by their dimensions. Since $gq/(bQ) < 1$, choose $0 < \alpha < 1$ with $gq/(bQ))^\alpha < 1$. Concavity and induction, using $\mathbb{E}(K+1)^\alpha \leq (1 + kq/(bQ))^\alpha$, give

$$f(k) = O((k+1)^\alpha). \tag{A.3}$$

The strict inequality absorbs C_0 for large k ; enlarge the inductive constant for the finite initial range. Neither α nor that constant is needed by the printer.

For successive $t \geq 1$, let $r_1 < \dots < r_t$ be the first t primes greater than $2q$, and put

$$m_t = \prod_{i=1}^t r_i, \quad \ell_t = m_t 2^{\lceil \log_2(16r_t) \rceil}, \quad R_t = \mathbb{Q}[z]/(\Phi_{\ell_t}(z)).$$

Cyclotomic polynomials and arithmetic in this finite algebra are effective: compute Φ_s recursively from $z^s - 1 = \prod_{d|s} \Phi_d(z)$, and use polynomial division and the Euclidean algorithm. A polynomial is a unit modulo Φ_s exactly when their gcd is 1. The distinct roots of Φ_s are the primitive s th roots, so identities in R_t can be checked at all primitive specializations. Roots of orders dividing ℓ_t are compatible powers of z . Carry out the next constructions over $E \otimes_{\mathbb{Q}} R_t$, with the Fourier coefficients in the central factor R_t . The two certificate identities and their inverse witnesses persist under extension of scalars; in particular, every certified unit remains a unit.

Use the exact-width Fourier layer construction of Lemma 3.5 on each width r_i . It gives $O(1 + \log^4 r_t)$ monomial or disjoint shear layers. Its rational formulas hold in R_t : all denominators are nonzero at every primitive specialization, and the convolution root orders divide the indicated power of two. In particular the Newton denominators $\prod_{a=1}^j (1 - u^a)$ have $j < r_i$ for a primitive r_i th root u . The measured workspace test is a test on the integer circuit description and is unchanged by this scalar extension.

To substitute the second word of (A.2), split any shear coefficient $c \in R_t$ into two units $d + (c - d)$, where d is a positive integer chosen by testing $d = 1, 2, \dots$ until $c - d$ is a unit. This terminates: only finitely many complex values of c at the primitive roots can be positive integers. A shear with a unit coefficient v is obtained by conjugating U by $\text{diag}(v, 1, \dots, 1)$, so it has the fixed word pattern from (A.2). The resulting word restores the arbitrary values of its $q - 2$ spectator coordinates. Partition the disjoint pairs of an axis into at most q batches of at most $\lfloor r_i/q \rfloor$ pairs; this is possible because $r_i > 2q$ gives $q \lfloor r_i/q \rfloor \geq \lfloor r_i/2 \rfloor$. Each batch fits disjoint q -tuples inside the same r_i coordinates. Its spectators may belong to other pairs, since they are restored before the next batch.

Synchronize these fixed word patterns across axes, with identities in idle positions. Their product is the tensor of the local products by the tensor multiplication identity. Each monomial slot has linear size. Every other slot is a direct sum of $A^{\otimes k}$ with $k \leq t$ and total width m_t , so (A.3) bounds its size by $O(m_t(t + 1)^\alpha)$. Chinese remainder reindexing gives F_{m_t} from the tensor of the local Fourier transforms. Now specialize R_t by $z \mapsto \zeta_{\ell_t}$, obtaining a circuit over $E \otimes_{\mathbb{Q}} \mathbb{C}$ whose output matrix is F_{m_t} times the algebra identity. Expand only the fixed algebra E by Remark A.3. Its dimension contributes a constant factor, whereas R_t has supplied polynomial expressions for the complex coefficients. The resulting complex linear circuit has size

$$s_t = O(m_t(t + 1)^\alpha(1 + \log^4(t + 2))) = o(m_t \log m_t). \quad (\text{A.4})$$

Here $r_t = O_q((t + 1)^2)$, by the elementary prime estimate in Lemma 5.1, and $\log m_t \geq t \log 2$. The printer computes all tensor coefficients and all wiring explicitly. It may take any finite amount of bit computation to do so. Each coefficient is a polynomial with rational coefficients evaluated at ζ_{ℓ_t} ; symbolic inverses have already been reduced in R_t . For each polynomial coefficient, print a finite Horner evaluation list, with its rational literals written explicitly; sharing previously computed values is optional. Thus the printed record contains a complete circuit and a fully expanded scalar preparation program, with no unspecified complex constants or hidden loops.

A.3 From a slow printer to a fast uniform algorithm

Theorem A.4 (Bounded-printer uniformization). *Suppose a fixed bit machine prints a sequence of complete records (m_t, C_t) with strictly increasing $m_t \rightarrow \infty$, where C_t computes F_{m_t} by $s_t = o(m_t \log m_t)$ scalar linear gates. Each record must explicitly list the gates, output indices, and a scalar preparation program with every instruction explicitly listed and executed once in order. Its instructions use rational literals, field operations with valid nonzero divisors, and canonical Fourier roots $\zeta_a = \exp(2\pi i/a)$ of positive integer orders whose orders are written literally in binary. Then*

there is one deterministic algorithm computing every F_n in $o(n \log n)$ time, including preparation and indexing in the model of this paper. A single specified root of order less than $8n^3$ suffices.

Proof. The printing budget will control both the circuit description and all literal scalar data, without requiring a bound on when a record appears. Normalize the printer to output at most one bit per machine step. For $n \geq 2$, run it from the beginning for $B = \lfloor \log_2 n \rfloor$ bit steps. Select the last completed record with $2 \leq m_t \leq \sqrt{B}$; if there is none, use the ordinary radix $m = 2$ circuit. Every fixed record eventually finishes and satisfies the width bound, so the selected stream index tends to infinity. In particular $m \rightarrow \infty$ and $s/(m \log m) \rightarrow 0$.

The total visible description has at most B bits. Decode the selected explicit lists to addressable arrays and evaluate its preparation program once. Construct rational literals by their binary expansions and perform each listed operation in the stated arithmetic model. This costs $B^{O(1)}$, including parsing and bit simulation. The gate list then executes on each new input in $O(m + s)$ time, including entry and output copies. There is no expansion of a succinct circuit during its invocation.

Put $X = 2n - 1$, let m^H be the largest power of m not exceeding X , and set

$$p = \lceil X/m^H \rceil, \quad P = m^H p.$$

Then $1 \leq p \leq m$, $X \leq P < 2X < 4n$, and $H \leq \log(2n)/\log m$. The mixed-radix Fourier recursion uses H copies of radix m and one radix p , the latter computed directly. For clarity, at a split ab use input index $bj_a + j_b$ and output index $ka + ak_b$. The identity

$$(bj_a + j_b)(ka + ak_b) \equiv bj_a k_a + aj_b k_b + j_b k_a \pmod{ab}$$

gives the two smaller Fourier transforms and their intervening twiddle. At each level, copying subarrays into the required order, tabulating or reading twiddles, and enumerating the two indices cost linear work. Reuse the decoded radix circuit each time. Tabulate powers of ζ_P in $O(P)$ operations; every recursive width divides P . Thus one transform, or its inverse by frequency reversal and scaling, costs

$$O(P[1 + p + H(1 + s/m)]) + B^{O(1)}. \tag{A.5}$$

All indices and products of indices have polynomial bounds in n , so a constant number of $O(\log(n+2))$ -bit words suffices per access.

Finally reduce F_n to convolution of length P . With $\eta = \zeta_{2n}$, store $\eta^{j^2} x_j$ for $0 \leq j < n$ and pad with zeros. The fixed kernel stores η^{-h^2} at the residues of $-(n-1) \leq h \leq n-1$; these residues are distinct since $P \geq 2n-1$. Multiply convolution output k by η^{k^2} . The identity $k^2 + j^2 - (k-j)^2 = 2kj$ gives exactly $F_n x$. Computing the fixed kernel's transform, the variable transform and the inverse uses only a constant number of transforms bounded by (A.5), plus linear preparation and scaling work. The chirp scalars come from an $O(n)$ -size table of powers of η , indexed by the squared exponents modulo $2n$.

For the root accounting, let R be the product of the root orders listed in the selected preparation program, or 1 if there are none. Their total binary length is at most B , whence $R \leq 2^B \leq n$. The single root ζ_D , where $D = 2nPR < 8n^3$, supplies every listed root and the two outer roots by integer powering. There are at most B such requests, and powering and forming their exponents cost $B^{O(1)}$. This includes obtaining the required powers of the supplied root in the preparation bound.

Since $p = O(1 + \sqrt{\log n})$, dividing the arithmetic term of (A.5) by $n \log n$ gives

$$O\left(\frac{1+p}{\log n} + \frac{1}{\log m} + \frac{s}{m \log m}\right) = o(1).$$

The polynomial logarithmic preparation term is also $o(n \log n)$. Length 1 is the identity. This proves the claim, with all choices independent of the input values. $\square$

Applying Theorem [A.4](#) to the printer above yields a uniform all-length $o(n \log n)$ algorithm directly from the finite certificate interface. The symbolic search, finite algebra, and stored circuits are implementation devices; the only supplied complex primitive is the explicitly specified Fourier root. This conclusion complements the explicit quantitative construction of the main text.

References

- [1] Nir Ailon. A lower bound for Fourier transform computation in a linear model over 2×2 unitary gates using matrix entropy. arXiv:1305.4745v1, 2013. Version of May 21, 2013.
- [2] Nir Ailon. An $\Omega((n \log n)/R)$ lower bound for Fourier transform computation in the R -well conditioned model. arXiv:1403.1307v5, 2014. Version of July 24, 2014.
- [3] Josh Alman and Kevin Rao. Faster Walsh–Hadamard and discrete Fourier transforms from matrix non-rigidity. In *Proceedings of the 55th Annual ACM Symposium on Theory of Computing*, pages 455–462. ACM, 2023. Extended version: arXiv:2211.06459v2.
- [4] Charles H. Bennett. Logical reversibility of computation. *IBM Journal of Research and Development*, 17(6):525–532, 1973.
- [5] L. I. Bluestein. A linear filtering approach to the computation of discrete Fourier transform. *IEEE Transactions on Audio and Electroacoustics*, 18(4):451–455, 1970.
- [6] Harry Buhrman, Richard Cleve, Michal Koucký, Bruno Loff, and Florian Speelman. Computing with a full memory: Catalytic space. In *Proceedings of the Forty-Sixth Annual ACM Symposium on Theory of Computing*, pages 857–866. ACM, 2014.
- [7] James W. Cooley, Peter A. W. Lewis, and Peter D. Welch. Historical notes on the fast Fourier transform. *Proceedings of the IEEE*, 55(10):1675–1677, 1967.
- [8] James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90):297–301, April 1965.
- [9] I. J. Good. The interaction algorithm and practical Fourier analysis. *Journal of the Royal Statistical Society, Series B (Methodological)*, 20(2):361–372, 1958.
- [10] Thomas Kailath, Sun-Yuan Kung, and Martin Morf. Displacement ranks of a matrix. *Bulletin of the American Mathematical Society (New Series)*, 1(5):769–773, 1979.
- [11] Jacques Morgenstern. Note on a lower bound on the linear complexity of the fast Fourier transform. *Journal of the ACM*, 20(2):305–306, April 1973.
- [12] OpenAI. Finite tensor savings and exact Fourier circuits. OpenAI Math Release preprint [OAI:Finite-tensor-savings-and-exact-Fourier-circuits-September-25-2026](#), 2026.
- [13] OpenAI. Integer multiplication below $n \log n$. OpenAI Math Release preprint [OAI:Integer-multiplication-below-n-log-n-September-23-2026](#), 2026.
- [14] Lawrence R. Rabiner, Ronald W. Schafer, and Charles M. Rader. The chirp z -transform algorithm. *IEEE Transactions on Audio and Electroacoustics*, 17(2):86–92, 1969.
- [15] The Stacks Project Authors. The stacks project: Nakayama’s lemma. <https://stacks.math.columbia.edu/tag/00DV>, 2026. Tag 00DV; accessed September 25, 2026.
- [16] The Stacks Project Authors. The stacks project: Noether normalization. <https://stacks.math.columbia.edu/tag/000Y>, 2026. Tags 051M, 051N, 00OX, and 00OY; accessed September 25, 2026.